

Команды Git

Основные команды

Настройка

```
# Установка имени пользователя
git config --global user.name "Ваше Имя"

# Установка email
git config --global user.email "ваш.email@example.com"

# Проверка настроек
git config --list
```

Создание и клонирование репозитория

```
# Инициализация нового репозитория
git init

# Клонирование существующего репозитория
git clone https://github.com/username/repository.git

# Клонирование определенной ветки
git clone -b branch-name https://github.com/username/repository.git
```

Основной рабочий процесс

```
# Проверка статуса репозитория
git status

# Добавление файлов в индекс
git add file.txt
git add *.js
git add .

# Создание коммита
git commit -m "Сообщение коммита"

# Добавление и коммит одной командой (для отслеживаемых файлов)
git commit -am "Сообщение коммита"

# Просмотр истории коммитов
git log
```

```
git log --oneline
git log --graph --oneline --all
```

Работа с удаленными репозиториями

```
# Добавление удаленного репозитория
git remote add origin https://github.com/username/repository.git

# Просмотр удаленных репозиториях
git remote -v

# Получение изменений без слияния
git fetch origin

# Получение изменений и слияние
git pull origin master

# Отправка изменений
git push origin master

# Установка отслеживания удаленной ветки
git push -u origin master
```

Продвинутые команды

Работа с ветками

```
# Создание новой ветки
git branch new-branch

# Переключение на ветку
git checkout branch-name

# Создание и переключение на новую ветку
git checkout -b new-branch

# Просмотр всех веток
git branch
git branch -a # включая удаленные ветки

# Удаление ветки
git branch -d branch-name
git branch -D branch-name # принудительное удаление

# Переименование ветки
git branch -m old-name new-name
```

Слияние и перебазирование

```
# Слияние ветки в текущую
git merge branch-name

# Перебазирование текущей ветки на другую
git rebase branch-name

# Интерактивное перебазирование
git rebase -i HEAD~3 # для последних 3 коммитов
```

Временное сохранение изменений (stash)

```
# Сохранение изменений во временное хранилище
git stash

# Сохранение с сообщением
git stash save "Сообщение"

# Просмотр списка сохранений
git stash list

# Применение последнего сохранения
git stash apply

# Применение конкретного сохранения
git stash apply stash@{2}

# Применение и удаление последнего сохранения
git stash pop

# Удаление последнего сохранения
git stash drop

# Удаление конкретного сохранения
git stash drop stash@{2}

# Очистка всех сохранений
git stash clear
```

Работа с тегами

```
# Создание легковесного тега
git tag v1.0.0

# Создание аннотированного тега
git tag -a v1.0.0 -m "Версия 1.0.0"
```

```
# Просмотр тегов
git tag

# Просмотр информации о теге
git show v1.0.0

# Отправка тегов в удаленный репозиторий
git push origin v1.0.0
git push origin --tags # отправка всех тегов
```

Отмена изменений

```
# Отмена изменений в рабочей директории
git checkout -- file.txt

# Отмена изменений в индексе
git reset HEAD file.txt

# Отмена последнего коммита с сохранением изменений
git reset --soft HEAD^

# Отмена последнего коммита без сохранения изменений
git reset --hard HEAD^

# Создание нового коммита, отменяющего изменения
git revert commit-hash
```

Просмотр изменений

```
# Просмотр изменений в рабочей директории
git diff

# Просмотр изменений в индексе
git diff --staged

# Просмотр изменений между коммитами
git diff commit1 commit2

# Просмотр изменений в файле между коммитами
git diff commit1 commit2 -- file.txt
```

Поиск и фильтрация

```
# Поиск коммитов по автору
git log --author="Имя"

# Поиск коммитов по сообщению
git log --grep="ключевое слово"

# Поиск коммитов, затрагивающих определенный файл
git log -- file.txt

# Поиск строки в истории
git log -S"строка для поиска"
```

Очистка и обслуживание

```
# Удаление неотслеживаемых файлов
git clean -f

# Удаление неотслеживаемых файлов и директорий
git clean -fd

# Интерактивная очистка
git clean -i

# Сжатие репозитория
git gc

# Проверка целостности репозитория
git fsck
```

Алиасы (сокращения команд)

Вы можете создать сокращения для часто используемых команд:

```
# Создание алиаса
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.ci commit
git config --global alias.st status

# Пример использования алиаса
git co master # эквивалентно git checkout master
```

Полезные однострочные команды

```
# Просмотр графика коммитов в одну строку
git log --pretty=format:"%h %ad | %s%d [%an]" --graph --date=short

# Поиск всех больших файлов в репозитории
git rev-list --objects --all | git cat-file --batch-check='%(objecttype) %
(objectname) %(objectsize) %(rest)' | awk '/^blob/ {print substr($0,6)}' | sort
-k2nr | head -n 10

# Список авторов с количеством коммитов
git shortlog -sn
```

Заключение

Это лишь часть команд Git. Для более глубокого изучения рекомендуется обратиться к официальной документации и практиковаться в реальных проектах.

Полезные ресурсы

- [Официальная документация Git](#)
- [Pro Git Book](#)
- [Git Cheat Sheet](#)
- [Интерактивное обучение Git](#)